Domain-Driven Design (**DDD**), Command Query Responsibility Segregation (**CQRS**)

****

Simple Object Access Protocol (**SOAP**)

****

Interface definition language (IDL)

****

What is gRPC?

One of the most popular RPC frameworks is gRPC. It is a high-performance, **open-source modern RPC framework** for building network services and distributed applications. gRPC was initially created by Google, which used a RPC framework called Stubby. In March 2015, Google decided to make it open-source, resulting in gRPC, which is now used in many organizations outside of Google.

---

gRPC has some awesome features, such as the following:

• **Interoperability**: gRPC uses a Protocol Buffer (protobuf) file to declare services and messages, which enables gRPC to be completely language- and platform-agnostic. You can find gRPC tools and libraries for all major programming languages and platforms.

• **Performance**: protobuf is a binary format, which has a smaller size and faster performance than JSON. It is not readable by humans, but it is readable by computers. HTTP/2 also supports multiplexing requests over a single connection. It needs fewer resources, even in slower networks.

• **Streaming**: gRPC is based on the HTTP/2 protocol, which makes it support ***bidirectional streaming***.

• **Productivity**: Developers author protobuf files (.proto) to describe services with input and output. Then, they can use the .proto files to generate stubs for different languages or platforms. It is similar to the OpenAPI specification. Teams can focus on business logic and work on the same service in parallel.

• **Security**: gRPC is designed to be secure. HTTP/2 is built on top of Transport Layer Security (TLS) end-to-end encrypted connection. It also supports client certificate authentication.

With these benefits, gRPC is a ***good choice for microservices-style architecture***. It can efficiently connect services in and across data centers, even ***across load balancers***. It is applicable in the last mile of distributed systems because service-to-service communication needs **low latency** and **high performance**. Also, the polyglot systems may have multiple languages or platforms, and gRPC can support different languages and platforms.

However, gRPC is not a silver bullet. There are several factors we need to consider before we choose gRPC:

• **Tight coupling due to protocol changes**: The client and server are tightly coupled because of the protocol. Once the protocol changes, the client and server must be updated, even just changing the order of the parameters.

• **Non-human readable format**: protobuf is a non-human readable format, so debugging is not convenient. Developers need additional tools to analyze the payloads.

• **Limited browser support**: gRPC heavily relies on HTTP/2, but browsers cannot support gRPC natively. That is why it is more suitable for service-to-service communication. There are some projects, such as **grpcweb**, that can provide a library to perform conversions between gRPC and HTTP/1.1.

• **No edge caching**: gRPC uses the POST method, which is not cacheable for the clients.

• **A steeper learning curve**: Unlike REST, which is human-readable, many teams find gRPC challenging to learn. They need to learn protobuf and HTTP/2 and look for proper tools to deal with the message content.

In a microservices architecture, the services are loosely coupled, and each one does a specific task or processes specific data. They need to be able to communicate with each other with simplicity and efficiency without considering browser compatibility. gRPC is suitable for this scenario because it is based on HTTP/2, which is a high-performance protocol and provides bidirectional streaming, binary messaging, and multiplexing. Dapr, which is a portable, event-driven runtime for microservices, implements gRPC APIs so that apps can communicate with each other via gRPC.

****

Besides solving the **over- and under-fetching problem**, GraphQL has some other advantages:

• **GraphQL reduces the complexity of maintaining API versions**. There is only one endpoint and one version of the graph. It allows the API to evolve without breaking the existing clients.

• **GraphQL uses a strong type system to define the types and fields in a schema using SDL**. The schema behaves as the contract, which reduces the miscommunication between the client and the server. Developers can develop frontend applications by mocking the required data structures. Once the server is ready, they can switch to the actual API.

• **GraphQL does not define the specific application architecture**, which means it can work on top of an existing REST API to reuse some code.

• **The payload is smaller** because clients get what they exactly request without over-fetching.

However, there are also some disadvantages of using GraphQL:

• ***GraphQL presents*** a high learning curve for REST API developers.

• ***The implementation of the server is more complicated***. The query could be complex.

• ***GraphQL uses a single endpoint***, which means it cannot leverage the full capabilities of HTTP. It does not support HTTP content negotiation for multiple media types beyond JSON.

• ***It is challenging to enforce the authorization*** because, normally, the API gateway enforces access control based on URLs. Rate-limiting is also often associated with the path and HTTP methods. So, you need more consideration to adopt the new style.

• ***Caching is complicated*** to implement because the service does not know what data clients need.

• ***File uploads*** are not allowed, so a separate API for file handling is needed.

****

Some technologies can implement real-time APIs, such as long polling, Server-Sent Events (SSE), WebSocket, SignalR, and gRPC streaming.

****

Uni-directional versus bidirectional Uni-directional communication is like a radio.

SSE is uni-directional because the server broadcasts data to clients, but clients cannot send data to the server. Bidirectional communication supports two-way communication.

There are two types of bidirectional communication — half-duplex and full-duplex.

Half-duplex communication is like a walkie-talkie. Both the server and client can send messages to each other, but only one party may send messages at a time.

Full-duplex communication is like a telephone. The message can be sent from either side at the same time. WebSocket is full-duplex.

****

gRPC takes advantage of the HTTP/2 protocol to support bidirectional communication. A gRPC service supports these streaming combinations:

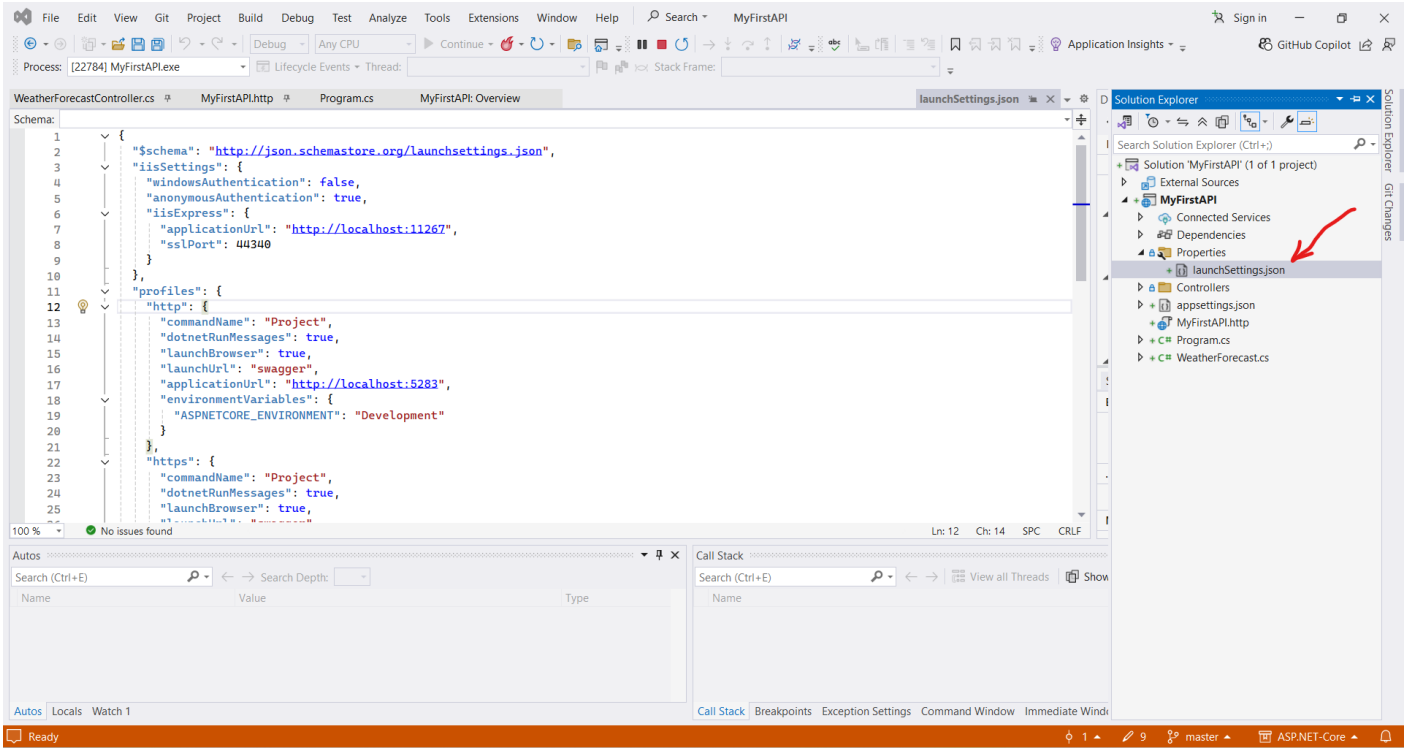
• Unary (No streaming)

• Server-to-client streaming

• Client-to-server streaming

• Bidirectional streaming

****



****

Some commands, such as **dotnet build**, **dotnet run**, **dotnet test**, and **dotnet publish**, will implicitly restore dependencies.

****

From version 5.0, ASP.NET Core enables OpenAPI support by default. It uses the Swashbuckle.AspNetCore NuGet package, which provides the Swagger UI to document and test the APIs.

****

Note (From me): When use http request, it will redirect to https request.

****

HttpRepl

HTTP Read-Eval-Print Loop (HttpRepl) is a command-line tool that allows you to test APIs. It is lightweight and cross-platform, so it can run anywhere. It supports the GET, POST, PUT, DELETE, HEAD, OPTIONS, and PATCH HTTP verbs. To install HttpRepl, you can use the following command:

dotnet tool install -g Microsoft.dotnet-httprepl

****

After the installation, you can use the following command to connect to our API: httprepl / is the base URL of the web API, such as the following:

httprepl http://localhost:5247/

****

After we invoke ls-command, to see the Folders that we have:

```
http://localhost:5247/> cd WeatherForecast
/WeatherForecast    [GET]
```

Then, we can use the `get` command to test the endpoint, such as the following:

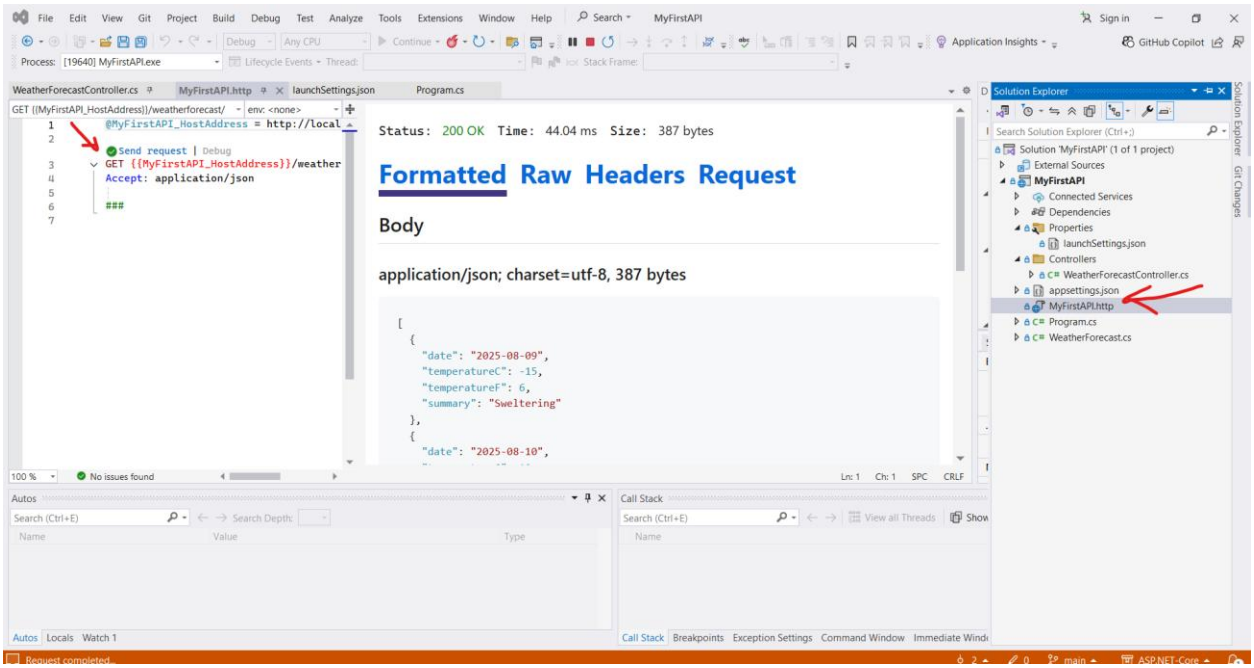
```
http://localhost:5247/WeatherForecast> get
```

****

Using .http files in VS 2022

If you use Visual Studio 2022, you can use the .http file to test the API. The .http file is a text file that contains definitions of HTTP requests. The latest ASP.NET Core 8 template project provides a default .http file. You can find it in the **MyFirstApi folder**.

****



****

An ASP.NET Core web API project follows the basic MVC pattern, but it does not have views, so it only has a Model layer and a Controller layer.

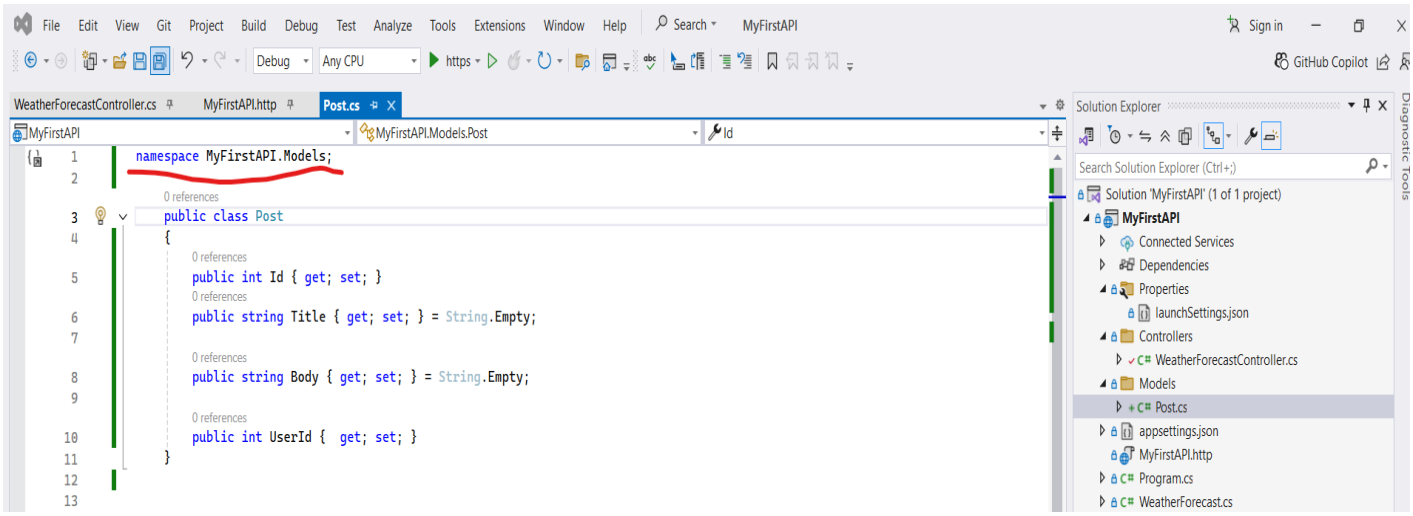
****

To avoid warning about NULL Values, we can initialize the Attributes:

```
public string Title { get; set; } = String.Empty;
public string Body { get; set; } = String.Empty;
```

****

Note: File-scoped namespace declaration From C# 10, you can use a new form of namespace declaration, as shown in the previous code snippet, which is called a file-scoped namespace declaration. All the members in this file are in the same namespace. It saves space and reduces indentation.



****

To create service that handle the logic of CRUD of specific Controller:

```
using MyFirstAPI.Models;

namespace MyFirstAPI.Services;
public class PostsService
{
    private static readonly List<Post> AllPosts = new List<Post>();

    public Task CreatePost(Post item)
    {
        AllPosts.Add(item);

        return Task.CompletedTask;

    }

    public Task<Post?> UpdatePost(int id, Post item)
    {
        var post = AllPosts.FirstOrDefault(x => x.Id == id);

        if (post != null)
        {
            post.Title = item.Title;
            post.Body = item.Body;
            post.UserId = item.UserId;
        }

        return Task.FromResult(post);
    }

    public Task<Post?> GetPost(int id)
    {
        return Task.FromResult(AllPosts.FirstOrDefault(x =>x.Id == id));
    }

    public Task<List<Post>> GetAllPosts()
    {
        return Task.FromResult(AllPosts);
    }

    public Task DeletePost(int id)
    {
        var post = AllPosts.FirstOrDefault(x => x.Id == id);
        if (post != null) {
            AllPosts.Remove(post);
        }

        return Task.CompletedTask;
    }
}
```

****

After we initialize the PostsService in Constructor:

```
[HttpGet("{id}")]
public async Task<ActionResult<Post>> GetPost(int id)
{
    var post = await _postsService.GetPost(id);

    if (post == null)
    {
        return NotFound();
    }

    return Ok(post);
}
```

****